



*Ingenieria Informatica*

Procesadores de Lenguaje 25/26

Grupo 81

*Practica Final - Segunda Parte*

«Traductor de un subconjunto de C a un  
lenguaje de notación prefija (Lisp)»

---

121:

Denis Loren Moldovan —

100522240@alumnos.uc3m.es

Jorge Adrian Saghin Dudulea —

100522257@alumnos.uc3m.es

*Profesor*

Maria Paz

## Tabla de contenidos

|                                |   |
|--------------------------------|---|
| 1. VARIABLES LOCALES .....     | 2 |
| 2. FOR LOOP .....              | 3 |
| 3. SWITCH/CASE .....           | 4 |
| 4. FUNCIONES (FUNCTIONS) ..... | 5 |
| 5. VECTORES (VECTORS) .....    | 6 |

## 1. VARIABLES LOCALES

Para implementar las variables locales, hemos añadido funciones y variables globales de c para poder detectar si estamos en una función y almacenar las variables locales. La variable local `en_funcion` indica 0 si estamos fuera de una función y 1 si estamos dentro. Para implementarlo, lo que hemos hecho ha sido añadir acciones semánticas que setean el valor de `en_funcion` a 1 despues de que se hayan parseado los tokens de “)” que contienen la lista de argumentos de las funciones. Y al acabar de parsear el cuerpo y el “}”, le asignamos el valor 0 a la funcion. También hemos implementado funciones que borran la lista de variables locales.

## 2. FOR LOOP

El bucle for de C se traduce descomponiéndolo en dos partes separadas de Common Lisp: la inicialización como una asignación independiente (setq/setf), y el resto como un loop while equivalente, donde el incremento/decremento se coloca al final del cuerpo del bucle.

La gramática acepta como inicialización tanto una asignación a variable ya existente como una declaración con tipo (int i = 0), en cuyo caso se registra la variable como local y se le añade el prefijo del nombre de la función para evitar colisiones con variables globales. Como operación de iteración solo se admiten inc(var) y dec(var), que se traducen a (setf var (+ var 1)) y (setf var (- var 1)) respectivamente.

Por ejemplo, for (int i = 0; i < 10; inc(i)) { x = x + i; } se traduce a:

```
(setq main_i 0)
(loop while (< main_i 10) do
  (setf main_x (+ main_x main_i))
  (setf main_i (+ main_i 1)))
```

### 3. SWITCH/CASE

El switch statement no tiene ninguna implementación complicada. Al igual que el resto de la gramática, simplemente hemos añadipardo el token para el SWITCH, y en cada caso se puede insertar el cuerpo completo, si al final encuentra el break.

Además, estos se pueden encadenar dentro del cuerpo, para añair mas switches en niveles superiores.

## 4. FUNCIONES (FUNCTIONS)

Las funciones de C se traducen a defun de Common Lisp. La gramática reconoce la cabecera con su lista de argumentos y el cuerpo con sus sentencias, generando directamente la forma (defun nombre (args) cuerpo).

Para gestionar las variables locales, cada vez que se entra en una función se limpia la tabla de variables locales (vars\_locales). Toda variable declarada dentro de la función se registra en esa tabla y se le añade el prefijo nombre\_funcion en el código generado, evitando así colisiones con variables globales del mismo nombre. Cuando se referencia un identificador, se comprueba si está en la tabla local y se aplica el prefijo correspondiente.

Los argumentos formales, en cambio, no se prefijan: se usan tal cual en la definición del defun, ya que Lisp los trata como variables locales al ámbito de la función de forma nativa.

El return se traduce a (return-from nombre\_funcion valor). Si el return es la última sentencia y no hay código posterior ni está dentro de una rama (nivel\_rama == 0), se omite el return-from y se deja la expresión directamente, aprovechando que Lisp devuelve implícitamente el valor de la última forma evaluada.

Por ejemplo:

```
int suma(int a, int b) {  
    int r = a + b;  
    return r;  
}
```

Se traduce a:

```
(defun suma (a b)  
  (setq suma_r (+ a b))  
  suma_r)
```

## 5. VECTORES (VECTORS)

Los vectores se pueden declarar y definir globalmente, usarse como operandos, y acceder y modificar dentro del cuerpo de las funciones.

Al igual que con las variables globales, los vectores también usan los mismo métodos para comprobar que esté dentro del scope de cada función.